Using PHP to Access Database

There are several ways to access a database in PHP, including:

1. MySQLi (Improved MySQL) extension: This extension is specifically designed for accessing the MySQL database server. It provides an object-oriented interface, as well as a procedural interface for working with the database.

2. PDO (PHP Data Objects) extension: This extension provides a common interface for accessing different types of databases, including MySQL, PostgreSQL, SQLite, Oracle, and Microsoft SQL Server, among others.

3. mysqli_query() function: This function is part of the MySQLi extension and allows you to execute a single SQL query and retrieve the result.

4. mysqli_prepare() function: This function is also part of the MySQLi extension and allows you to prepare and execute a SQL statement multiple times with different parameters.

5. SQLite: This is a compact, self-contained, file-based database system that provides a simple way to store and retrieve data in PHP.

6. ODBC (Open Database Connectivity): This is a standard interface for accessing different types of databases, including MySQL, Microsoft SQL Server, Oracle, and others.

Each of these methods has its own advantages and disadvantages, and the best method to use depends on the specific requirements of your project. You can choose the method that best fits your needs, depending on the type of database you are using, the amount of data you need to store, and the complexity of your application.

PHP 5 and later can work with a MySQL database using:

- **MySQLi extension** (the "i" stands for improved)
- **PDO (PHP Data Objects)**

Earlier versions of PHP used the MySQL extension. However, this extension was deprecated in 2012.

Should I Use MySQLi or PDO?

1. If you need a short answer, it would be "Whatever you like".
2. Both MySQLi and PDO have their advantages:
3. PDO will work on 12 different database systems, whereas MySQLi will only work with MySQL databases.

4. So, if you have to switch your project to use another database, PDO makes the process easy. You only have to change the connection string and a few queries. With MySQLi, you will need to rewrite the entire code - queries included.
5. Both are object-oriented, but MySQLi also offers a procedural API.
6. Both support Prepared Statements. Prepared Statements protect from SQL injection, and are very important for web application security.

MySQL Examples in Both MySQLi and PDO Syntax

In this, and in the following chapters we demonstrate three ways of working with PHP and MySQL:

- MySQLi (object-oriented)
- MySQLi (procedural)
- PDO

1.MySQLi Installation

For Linux and Windows: The MySQLi extension is automatically installed in most cases, when php5 mysql package is installed.

For installation details, go to: http://php.net/manual/en/mysqli.installation.php

2.PDO Installation

For installation details, go to: http://php.net/manual/en/pdo.installation.php

To access a database using PHP, you need to use the following steps:

1.Connect to the database: You can use the mysqli_connect function to connect to the database. The function takes four parameters: hostname, username, password, and database name.

```
$host = "hostname";
$user = "username";
$password = "password";
$dbname = "database_name";

// Create connection
$conn = mysqli_connect($host, $user, $password, $dbname);

// Check connection
if (!$conn) {
```

```
    die("Connection failed: " . mysqli_connect_error());
}
```

2.Execute a SQL query: Once you have established a connection, you can execute a SQL query to retrieve or manipulate data in the database. You can use the mysqli_query function to execute a query. The function takes two parameters: the connection and the query string.

```
$sql = "SELECT column1, column2, column3 FROM table_name";
$result = mysqli_query($conn, $sql);
```

3.Process the results: The results of the query can be processed using a variety of functions. For example, you can use the mysqli_fetch_assoc function to return an associative array for each row.

```
if (mysqli_num_rows($result) > 0) {
    // output data of each row
    while($row = mysqli_fetch_assoc($result)) {
        echo "column1: " . $row["column1"]. " - column2: " . $row["column2"]. " - column3: " .
$row["column3"]. "<br>";
    }
} else {
    echo "0 results";
}
```

4.Close the connection: Finally, you should close the connection to the database when you are done using it. You can use the mysqli_close function to close the connection.

```
mysqli_close($conn);
```

This is a basic example of how to access a database using PHP. You can build on this example to create more complex applications that interact with a database.

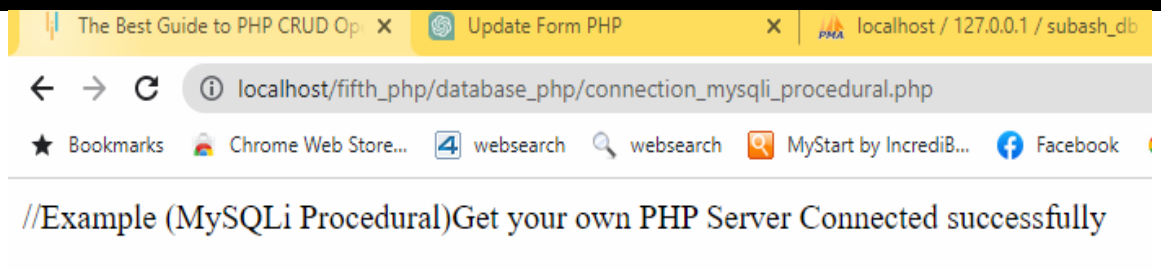
Way 1:

```
//Example (MySQLi Procedural)Get your own
PHP Server
<?php
$servername = "localhost";
$username = "root";
$password = "";

// Create connection
```

```
$conn = mysqli_connect($servername,
$username, $password);

// Check connection
if (!$conn) {
  die("Connection failed: " .
mysqli_connect_error());
}
echo "Connected successfully";
mysqli_close($conn);
?>
```



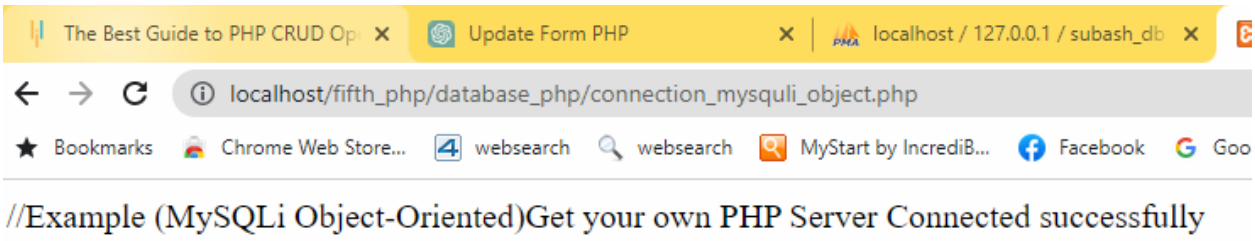
Way 2:

```
//Example (MySQLi Object-Oriented)Get your
own PHP Server
<?php
$servername = "localhost";
$username = "root";
$password = "";

// Create connection
```

```
$conn = new mysqli($servername, $username,
$password);

// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn-
>connect_error);
}
echo "Connected successfully";
$conn->close();
?>
```

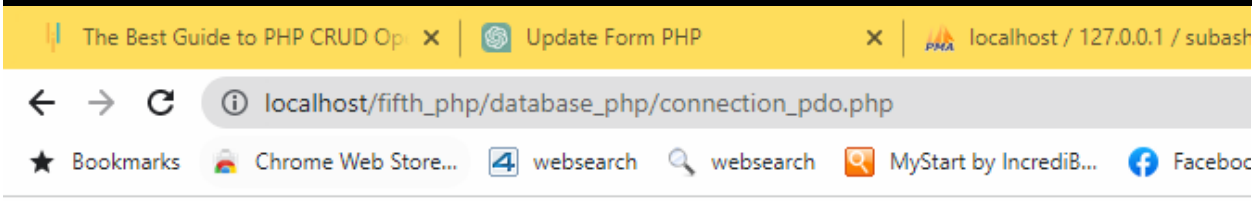


Way 3:

```
//Example (PDO)Get your own PHP Server
<?php
$servername = "localhost";
$username = "root";
$password = "";

try {
```

```
  $conn = new
PDO("mysql:host=$servername;dbname=subash_d
b", $username, $password);
  // set the PDO error mode to exception
  $conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
  echo "Connected successfully";
} catch(PDOException $e) {
  echo "Connection failed: " . $e-
>getMessage();
}
$conn = null;
?>
```



//Example (PDO)Get your own PHP Server Connected successfully

Here's an example of how you can connect to a database and retrieve data using PHP and MySQLi:

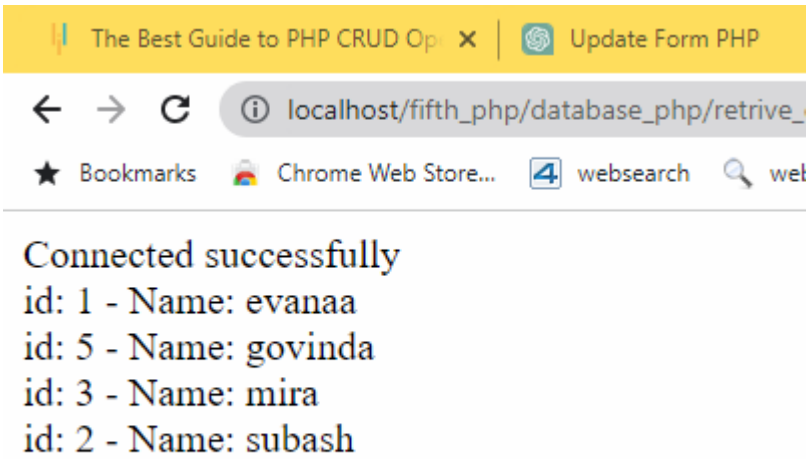
```
<?php
// Connect to the database
$conn = mysqli_connect("localhost", "root",
"", "subash_db");
```

```
// Check the connection
if (!$conn) {
    die("Connection failed: " .
mysqli_connect_error());
}
echo "Connected successfully"."<br>";

// Execute a SELECT query
$result = mysqli_query($conn, "SELECT *
FROM users");

// Process the result
if (mysqli_num_rows($result) > 0) {
    // Output the data
    while($row =
mysqli_fetch_assoc($result)) {
        echo "id: " . $row["id"]. " - Name:
" . $row["firstname"]. "<br>";
    }
} else {
    echo "No data found";
}
```

```
// Close the connection
mysqli_close($conn);
?>
```

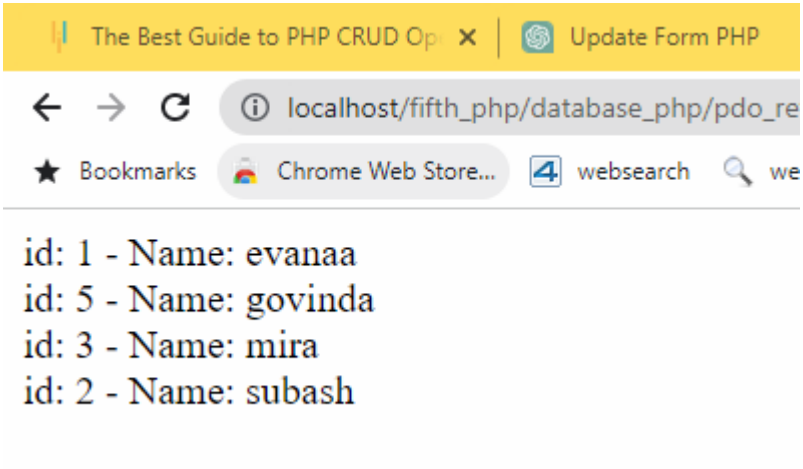


Similarly, you can use PDO to access the database as well. Here's an example:

```
<?php
// Connect to the database
$conn = new
PDO("mysql:host=localhost;dbname=subash_db"
, "root", "");

// Execute a SELECT query
$result = $conn->query("SELECT * FROM
users");
```

```php
// Process the result
if ($result->rowCount() > 0) {
    // Output the data
    while($row = $result-
>fetch(PDO::FETCH_ASSOC)) {
        echo "id: " . $row["id"]. " - Name:
" . $row["firstname"]. "<br>";
    }
} else {
    echo "No data found";
}
?>
```



Note that the above code is just a basic example to give you an idea of how to access a database using PHP. You should secure your code before using it in a production environment by escaping user input, using prepared statements, and other security measures.

Querying a Database with PHP

PHP is a server-side scripting language that is often used for web development. You can use PHP to query a database and retrieve information from it. Here is a simple example that demonstrates how to connect to a database and perform a SELECT query to retrieve data from a table:

```
<?php

$host = "hostname";
$user = "username";
$password = "password";
$dbname = "database_name";

// Create connection
$conn = mysqli_connect($host, $user, $password, $dbname);

// Check connection
if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}

$sql = "SELECT column1, column2, column3 FROM table_name";
$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) > 0) {
   // output data of each row
   while($row = mysqli_fetch_assoc($result)) {
      echo "column1: " . $row["column1"]. " - column2: " . $row["column2"]. " - column3: " .
$row["column3"]. "<br>";
   }
} else {
   echo "0 results";
}

mysqli_close($conn);

?>
```

In this example, the mysqli_connect function is used to connect to the database. You need to provide the hostname, username, password, and database name as arguments. If the connection is successful, mysqli_query function is used to execute a SQL query and retrieve the results. The retrieved data is then processed using the mysqli_fetch_assoc function, which returns an associative array for each row. Finally, the mysqli_close function is used to close the connection to the database.

CRUD Operations Using Forms in php

Here is an example of how you can implement CRUD operations using forms in PHP:

Example 1:

1.Create:

```
<form action="create.php" method="post">
  <input type="text" name="name" placeholder="Name">
  <input type="email" name="email" placeholder="Email">
  <input type="submit" value="Submit">
</form>

<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  $name = $_POST['name'];
  $email = $_POST['email'];

  $conn = mysqli_connect('host', 'username', 'password', 'database');

  $query = "INSERT INTO users (name, email) VALUES ('$name', '$email')";
  $result = mysqli_query($conn, $query);

  if ($result) {
    echo "Record created successfully.";
  } else {
    echo "Error: " . $query . "<br>" . mysqli_error($conn);
  }

  mysqli_close($conn);
}
```

Read:

php

Copy code

```
<?php
$conn = mysqli_connect('host', 'username', 'password', 'database');

$query = "SELECT * FROM users";
$result = mysqli_query($conn, $query);

echo "<table>";
while ($row = mysqli_fetch_assoc($result)) {
  echo "<tr>";
  echo "<td>" . $row['name'] . "</td>";
  echo "<td>" . $row['email'] . "</td>";
  echo "</tr>";
```

```
}
echo "</table>";

mysqli_close($conn);
```

Update:

php

Copy code

```
<form action="update.php" method="post">
  <input type="hidden" name="id" value="<?php echo $id; ?>">
  <input type="text" name="name" placeholder="Name" value="<?php echo $name; ?>">
  <input type="email" name="email" placeholder="Email" value="<?php echo $email; ?>">
  <input type="submit" value="Submit">
</form>

<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  $id = $_POST['id'];
  $name = $_POST['name'];
  $email = $_POST['email'];

  $conn = mysqli_connect('host', 'username', 'password', 'database');

  $query = "UPDATE users SET name='$name', email='$email' WHERE id=$id";
  $result = mysqli_query($conn, $query);

  if ($result) {
    echo "Record updated successfully.";
  } else {
    echo "Error: " . $query . "<br>" . mysqli_error($conn);
  }

  mysqli_close($conn);
}
```

Delete:

php

Copy code

```
<form action="delete.php" method="post">
  <input type="hidden" name="id" value="<?php echo $id; ?>">
  <input type="submit" value="Delete">
```

Example 2:

CRUD Operations are typically performed on databases, hence, in this PHP CRUD Operations tutorial, you will implement CRUD techniques on MySQL databases with the help of PHP.

The CRUD acronym comprises all the major operations that are performed on a relational database. It stands for:

C = Create

R = Read

U = Update

D = Delete

1.How to Create a MySQL Database Connection?

First, create a connection between the database and your PHP code.

The following code acts as the connection between the webpage and the database where the data from the webpage will be stored.

Here, name the file as config.php

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "subash_db";
$conn = new mysqli($servername, $username,
$password, $dbname);
```

```
if ($conn->connect_error) {
    die("Connection failed: " . $conn-
>connect_error);
}
//else{
    //echo "connection successfully";
     //}
?>
```

2.How to Create Records?

The first operation in PHP CRUD Operations, Create, is responsible for creating tables or new records into an existing table. To do that, first, you must write the code for the webpage to create an entry in the database.
Name the file as create.php.

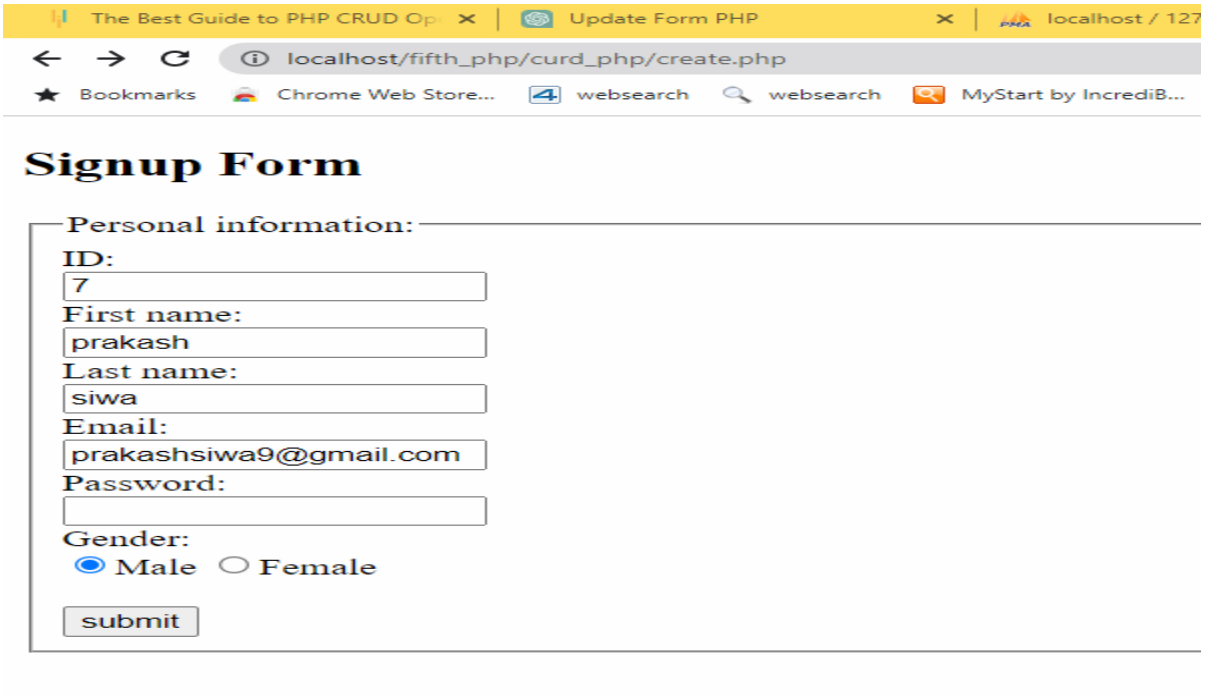
```
<?php
include "config.php";
  if (isset($_POST['submit'])) {
    $id = $_POST['id'];
    $first_name = $_POST['firstname'];
    $last_name = $_POST['lastname'];
    $email = $_POST['email'];
    $password = $_POST['password'];
    $gender = $_POST['gender'];
    $sql = "INSERT INTO
`users`(`id`,`firstname`, `lastname`,
```

```
`email`, `password`, `gender`) VALUES
('$id','$first_name','$last_name','$email',
'$password','$gender')";
    $result = $conn->query($sql);
    if ($result == TRUE) {
      echo "New record created
successfully.";
    }else{
      echo "Error:". $sql . "<br>". $conn-
>error;
    }
    $conn->close();
  }
?>
<!DOCTYPE html>
<html>
<body>
<h2>Signup Form</h2>
<form action="" method="POST">
  <fieldset>
    <legend>Personal information:</legend>
    ID:<br>
    <input type="text" name="id">
    <br>
```

```
    First name:<br>
    <input type="text" name="firstname">
    <br>
    Last name:<br>
    <input type="text" name="lastname">
    <br>
    Email:<br>
    <input type="email" name="email">
    <br>
    Password:<br>
    <input type="password" name="password">
    <br>
    Gender:<br>
    <input type="radio" name="gender"
value="Male">Male
    <input type="radio" name="gender"
value="Female">Female
    <br><br>
    <input type="submit" name="submit"
value="submit">
  </fieldset>
</form>
</body>
```

```
</html>
```

This page displays a signup form that stores the details entered on the page into the table named ‘users’.



3.How to Read/View Records?

The second operation, just as the name suggests, ‘Read’ is used to display or read the data that is already available in the database.

To perform the operation, you need to create a page that displays the records from the table ‘users’.

Now, name the page as read.php

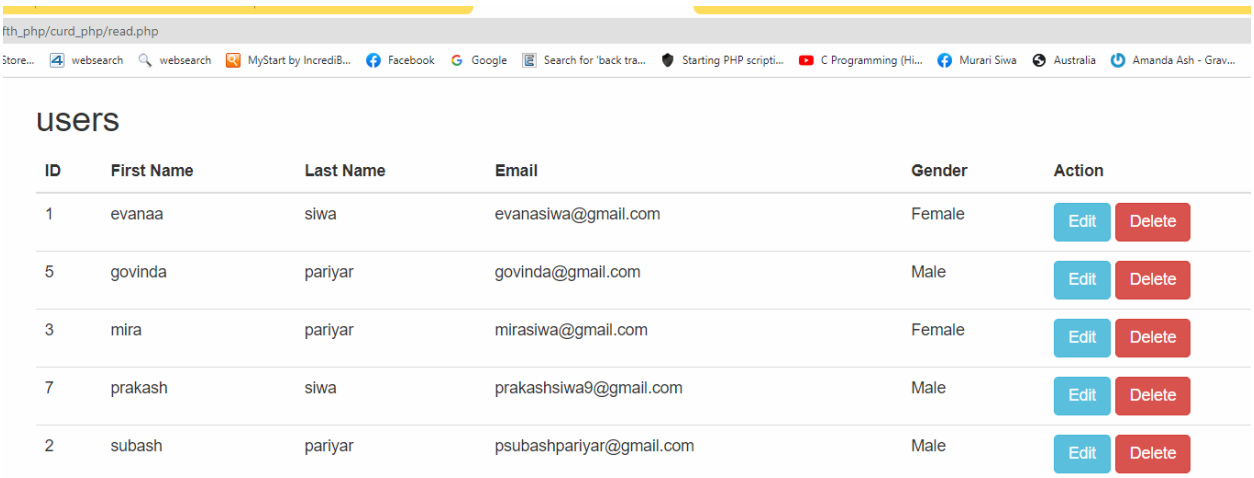
```
<?php
include "config.php";
$sql = "SELECT * FROM users";
$result = $conn->query($sql);
?>
<!DOCTYPE html>
<html>
```

```
<head>
    <title>Read Page</title>
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/boots
trap/3.4.0/css/bootstrap.min.css">
</head>
<body>
    <div class="container">
        <h2>users</h2>
<table class="table">
    <thead>
        <tr>
        <th>ID</th>
        <th>First Name</th>
        <th>Last Name</th>
        <th>Email</th>
        <th>Gender</th>
        <th>Action</th>
    </tr>
    </thead>
    <tbody>
        <?php
            if ($result->num_rows > 0) {
```

```
                while ($row = $result-
>fetch_assoc()) {
        ?>
                    <tr>
                    <td><?php echo
$row['id']; ?></td>
                    <td><?php echo
$row['firstname']; ?></td>
                    <td><?php echo
$row['lastname']; ?></td>
                    <td><?php echo
$row['email']; ?></td>
                    <td><?php echo
$row['gender']; ?></td>
                    <td><a class="btn btn-
info" href="update.php?id=<?php echo
$row['id']; ?>">Edit</a> <a class="btn
btn-danger" href="delete.php?id=<?php echo
$row['id']; ?>">Delete</a></td>
                    </tr>

        <?php       }
            }
        ?>
```

```
    </tbody>
</table>
    </div>
</body>
</html>
```



4.How to Update Records?

The third operation i.e, 'update' is used to change or modify the already existing data present in the database.

To do this, you need to create another page to update the details in the database.

Here, name the page as update.php

```
<?php
include "config.php";
    if (isset($_POST['update'])) {
        $firstname = $_POST['firstname'];
        $user_id = $_POST['user_id'];
        $lastname = $_POST['lastname'];
        $email = $_POST['email'];
```

```
        $password = $_POST['password'];
        $gender = $_POST['gender'];
        $sql = "UPDATE `users` SET
`firstname`='$firstname',`lastname`='$lastn
ame',`email`='$email',`password`='$password
',`gender`='$gender' WHERE
`id`='$user_id'";
        $result = $conn->query($sql);
        if ($result == TRUE) {
            echo "Record updated
successfully.";
        }else{
            echo "Error:" . $sql . "<br>" .
$conn->error;
        }
    }
if (isset($_GET['id'])) {
    $user_id = $_GET['id'];
    $sql = "SELECT * FROM `users` WHERE
`id`='$user_id'";
    $result = $conn->query($sql);
    if ($result->num_rows > 0) {
        while ($row = $result-
>fetch_assoc()) {
```

```
            $first_name =
$row['firstname'];
            $lastname = $row['lastname'];
            $email = $row['email'];
            $password  = $row['password'];
            $gender = $row['gender'];
            $id = $row['id'];
        }
    ?>
        <h2>User Update Form</h2>
        <form action="" method="post">
          <fieldset>
            <legend>Personal
information:</legend>
            First name:<br>
            <input type="text"
name="firstname" value="<?php echo
$first_name; ?>">
            <input type="hidden"
name="user_id" value="<?php echo $id; ?>">
            <br>
            Last name:<br>
```

```
            <input type="text"
name="lastname" value="<?php echo
$lastname; ?>">
            <br>
            Email:<br>
            <input type="email"
name="email" value="<?php echo $email; ?>">
            <br>
            Password:<br>
            <input type="password"
name="password" value="<?php echo
$password; ?>">
            <br>
            Gender:<br>
            <input type="radio"
name="gender" value="Male" <?php if($gender
== 'Male'){ echo "checked";} ?> >Male
            <input type="radio"
name="gender" value="Female" <?php
if($gender == 'Female'){ echo "checked";}
?>>Female
            <br><br>
            <input type="submit"
value="Update" name="update">
```

```
            </fieldset>
        </form>
        </body>
        </html>
    <?php
    } else{
        header('Location: read.php');
    }
}
?>
```

The Best Guide to PHP CRUD Op | Update Form PHP | localhost / 127.0.0.1 / subash_db

localhost/fifth_php/curd_php/update.php?id=1

Bookmarks | Chrome Web Store... | websearch | websearch | MyStart by IncrediB... | Facebook

User Update Form

Personal information:

First name:
evan

Last name:
siwa

Email:
evanasiwa@gmail.com

Password:
•••••••

Gender:
○ Male ◉ Female

Update

In the update form, we need to select the user id which we want to update. You can notice that the user id being updated is visible in the URL of the update page in the picture below.

You can view the updated details by opening the read.php webpage.

5.How to Delete Records?

The last operation of CRUD is Delete and just as the name suggests, it is used to delete an existing entry or table.
To perform this operation, you must create a page that would let you choose the data entry that you want to delete from the database.
Now, name the file delete.php

```
<?php
include "config.php";
if (isset($_GET['id'])) {
    $user_id = $_GET['id'];
    $sql = "DELETE FROM `users` WHERE
`id`='$user_id'";
     $result = $conn->query($sql);
     if ($result == TRUE) {
        echo "Record deleted
successfully.";
    }else{
        echo "Error:" . $sql . "<br>" .
$conn->error;
    }
}
?>
```

The Best Guide to PHP CRUD Op | Update Form PHP | localhost / 127.0.0.1 /

localhost/fifth_php/curd_php/delete.php?id=2

Bookmarks | Chrome Web Store... | websearch | websearch | MyStart by IncrediB... | F

Record deleted successfully.

/fifth_php/curd_php/read.php

b Store... | websearch | websearch | MyStart by IncrediB... | Facebook | Google | Search for 'back tra... | Starting PHP scripti... | C Programming (Hi... | Murari Siwa | Australia | Amanda Ash - Grav...

users

ID	First Name	Last Name	Email	Gender	Action
1	evanaa	siwa	evanasiwa@gmail.com	Female	Edit Delete
5	govinda	pariyar	govinda@gmail.com	Male	Edit Delete
3	mira	pariyar	mirasiwa@gmail.com	Female	Edit Delete
7	prakash	siwa	prakashsiwa9@gmail.com	Male	Edit Delete